Encrypted Cloud Storage with Searchable Symmetric Encryption

Vijayakumar P, Professor, SENSE

Shravan Adash M, 23BEC1149

Preetish RK, 23BEC1470

Prajinesh P, 23BEC1113

Abstract: The increasing adoption of cloud storage has created a conflict between protecting the confidentiality of outsourced data and retrieving that data efficiently. Files can be encrypted before they are uploaded, but encryption prevents conventional keyword search and forces users to download and decrypt their data before searching it. In order to overcome this limitation, this work presents the design, implementation and evaluation of an encrypted cloud storage system based on Searchable Symmetric Encryption (SSE). The system is built as two separate processes: a trusted client that holds every key, and an untrusted cloud service that stores and searches only ciphertext. Keys are derived from the user password with Argon2id and separated with HKDF-SHA256; documents and filenames are protected with AES-256-GCM; and a static encrypted inverted index following the basic construction of Cash et al. is built with HMAC-SHA256 so that the cloud can answer a keyword query from an opaque search token without learning the keyword or the document contents. Conjunctive (AND) multi-keyword search, document deletion and password rotation with full re-encryption are also supported. The system is evaluated with a reproducible benchmark suite covering cryptographic cost, index construction, search latency, index rebuild cost and storage overhead, and its information leakage (search pattern, access pattern and index size) is demonstrated from captured client–cloud traffic rather than only stated.

Keywords: Searchable symmetric encryption (SSE); encrypted cloud storage; encrypted inverted index; AES-256-GCM; HMAC-SHA256; Argon2id; leakage analysis

I. INTRODUCTION

Cloud storage has become a standard component of modern information systems because it offers scalability, availability and reduced infrastructure cost. Organisations and individuals routinely store documents and other sensitive information on third-party servers. However, outsourcing data in this way means that the owner no longer controls the infrastructure that holds it, so a curious or compromised provider, an insider, or an attacker who obtains the storage can read everything that is stored in plaintext. Conventional cryptographic mechanisms such as the Advanced Encryption Standard (AES) can encrypt the data before it is uploaded, which protects its confidentiality, but this introduces a serious usability problem: the server can no longer inspect the content, so keyword search cannot be performed on the server side. The user must retrieve and decrypt the data before every search, which becomes impractical as the volume of stored information grows. Searchable Symmetric Encryption (SSE) addresses this conflict between confidentiality and searchability. In an SSE system the data owner encrypts the documents and builds an encrypted index before outsourcing them; to search, the owner derives a cryptographic search token from the keyword and a secret key, and the server uses the token to locate the matching encrypted documents without ever seeing the keyword or the plaintext. Practical SSE systems must nevertheless address secure index construction, efficient token generation, search efficiency, data updates, and the limited information that every efficient SSE scheme reveals through its search and access patterns. Recent research has concentrated on dynamic updates, permission control, multi-keyword queries, forward and backward privacy, efficient deletion and scalable index structures, whereas many evaluations remain at the level of the cryptographic scheme rather than a complete running system. The proposed work implements an end-to-end encrypted cloud storage system in which the trust boundary between a trusted client and an untrusted cloud is a real process boundary, measures its performance with a reproducible benchmark suite, and demonstrates its information leakage from captured traffic. The organisation of the paper is as follows. Section I introduces the problem and the motivation for this work. Section II discusses related journal literature on searchable symmetric encryption. Section III describes the proposed system architecture, its cryptographic modules and the complete algorithm.

Section IV presents the implementation results and the performance evaluation. Section V concludes the paper with key observations and future research directions.

II. RELATED WORK

Hu et al. [1] proposed SEAC, a dynamic searchable symmetric encryption scheme with lightweight update–search permission control for cloud-assisted sharing of electronic protected health information. The scheme attaches permission information to secure indexes so that only authorised clients can perform particular search and update operations on the encrypted data, demonstrating that practical SSE requires secure permission management in addition to encrypted search. However, the work targets a multi-client healthcare setting with considerably more protocol machinery, and it does not address how the trust boundary between the data owner and the server is enforced in a running single-owner storage system. Xu et al. [2] proposed a forward-secure, multi-user and verifiable dynamic searchable encryption scheme for cloud storage in a zero-trust environment, in which the server is assumed to be potentially malicious. The scheme uses oblivious pseudorandom functions and hash-based state structures to protect search operations and updates and to let users verify the returned results. Although this demonstrates the importance of protecting the search process itself, the complexity of multi-user verifiable mechanisms makes such schemes better suited to advanced deployments than to a first practical implementation whose security properties must be fully explainable and testable.

Yang et al. [3] investigated dynamic searchable symmetric encryption with efficient and complete access control for multi-user cloud computing, addressing the limitation that many dynamic SSE schemes cannot enforce access control on both keyword retrieval and encrypted-data updates at the same time. The work shows that searchable encryption and authorisation must be combined in multi-user environments; however, access control across users is outside the single-owner scope of the proposed system and is identified here as an extension. A 2025 study [4] proposed a forward and backward private conjunctive dynamic SSE scheme with a refined leakage function and low communication. Using a three-dimensional chain structure, it supports searches over several keywords simultaneously while reducing the information revealed through updates and searches, and it reports significant reductions in communication cost for multi-keyword queries. This work is directly relevant because the proposed system also supports conjunctive queries, but does so by intersecting independent single-keyword results on the client, which is simpler and leaks more, as discussed in Section III.

Ren et al. [5] proposed a dynamic SSE scheme supporting efficient conjunctive queries and non-interactive real deletion. A cuckoo filter improves the efficiency of conjunctive queries, while separate search databases allow deleted ciphertexts to be physically removed. The study shows that practical searchable encryption must consider secure deletion as well as search. In the proposed system deletion is supported by rebuilding the encrypted index without the deleted document before the ciphertext is removed, which is simple and correct for a static index but has a cost that grows with the size of the vault. Zhang et al. [6] proposed a Bloom-filter-based dynamic symmetric searchable encryption scheme for cloud data that combines a B+-tree reverse index with Bloom filters to improve index construction and search efficiency while supporting dynamic updates. The work emphasises that the encrypted index structure determines search efficiency for larger datasets; its evaluation, however, concerns the index structure rather than the end-to-end latency a user experiences, which in a client–cloud system can be dominated by network round trips rather than by the index itself.

Wang et al. [7] proposed a decentralised multi-client Boolean keyword search system for encrypted cloud storage that removes the key-escrow problem of multi-client SSE and supports Boolean queries with sub-linear search complexity, evaluated on the Enron dataset. Wu et al. [8] proposed MMKFB, a multi-client and multi-keyword SSE scheme with forward and backward privacy that supports multi-keyword threshold queries using additive homomorphism and is particularly efficient when many files are updated at once. Both works represent an advanced direction involving decentralised key management, multiple clients and richer query types, and they show that supporting several keywords and several clients simultaneously requires substantial additional cryptographic machinery beyond a basic single-keyword construction. They do not, however, provide the kind of minimal, fully explainable baseline that is needed to measure where the practical costs of an SSE system actually arise.

Wu et al. [9] proposed an efficient verifiable searchable encryption scheme that conceals both the search pattern and the access pattern, using random values to obfuscate polynomial evaluations and thereby avoiding expensive ciphertext multiplications, while also allowing the user to verify that the returned results are complete and correct. This work is directly relevant because search pattern and access pattern are exactly the leakages retained by the proposed system; concealing them is identified here as future work rather than claimed. Khan et al. [10] proposed BloomSec, a scalable privacy-preserving searchable encryption scheme that stores an individual Bloom filter for each file rather than one consolidated keyword index, reducing the communication overhead of transmitting large lookup structures. The study shows that the representation chosen for the encrypted index directly affects both search efficiency and communication cost, which is consistent with the storage overhead measured for the encrypted index in Section IV of this work.

Xue et al. [11] proposed an incentive-assisted multi-keyword ranked searchable encryption scheme with hidden attributes for community data sharing, combining ranked search, fine-grained access control and verifiable results in a peer-to-peer setting together with a points-based incentive mechanism. The scheme illustrates how searchable encryption can be extended with ranking, access control and participation incentives, all of which lie outside the single-owner scope of the proposed system. Mohammed et al. [12] reviewed searchable symmetric encryption functionalities, challenges and solutions, surveying approximately eighty schemes covering single-keyword, multi-keyword, fuzzy, ranked, semantic and dynamic search and identifying the privacy gaps that remain in current implementations. The review provides a useful classification of the design space and confirms that single-keyword static SSE remains the baseline from which the more advanced functionalities are derived. Cash et al. [13] introduced the basic static construction Π_{bas} on which the encrypted index of the proposed system is based.

From the reviewed 2025–2026 literature, it is evident that recent SSE research concentrates on extending the scheme itself — with dynamic updates, forward and backward privacy, multi-client authorisation, conjunctive and Boolean queries, verifiability and improved index structures — and evaluates these extensions mainly through the cost of their cryptographic operations and index structures. Comparatively little attention is given to a complete running system in which the trust boundary between owner and server is enforced in practice, to where end-to-end latency is actually spent once network requests are included, and to demonstrating the scheme's leakage from real traffic rather than stating it formally. The proposed work addresses this gap by implementing an end-to-end SSE-based encrypted cloud storage system with a process-level trust boundary, measuring its cryptographic, indexing, search, rebuild and storage costs with a reproducible benchmark suite, and capturing client–cloud traffic to demonstrate exactly what the untrusted cloud can and cannot observe.

III. PROPOSED SSE-BASED ENCRYPTED CLOUD STORAGE SYSTEM

The proposed system is an encrypted cloud storage architecture that allows an authorised user to store documents on an untrusted server and later search them by keyword without the server learning the documents, their filenames or the searched keywords. It is implemented as two genuinely separate operating-system processes: a trusted client application on port 5000, which holds all key material and performs every cryptographic operation, and an untrusted cloud service on port 5001, which stores ciphertext and answers search requests. The web browser communicates only with the trusted client and never holds a key. When the vault is unlocked, the user password is stretched with Argon2id into a master key, which HKDF-SHA256 separates into a file-encryption key and an index key. Each uploaded document is converted to text, tokenised into a keyword set, and encrypted together with its filename using AES-256-GCM. An encrypted inverted index is then constructed with HMAC-SHA256 as a pseudorandom function, following the basic static construction Π_{bas} of Cash et al. [13], and replaced atomically on the cloud. To search, the client sends only a per-keyword search token; the cloud regenerates the index labels from the token and returns the matching encrypted identifiers, which the client decrypts locally. Multi-keyword queries are answered by intersecting single-keyword results on the client. The system therefore integrates password-based key management, authenticated encryption, encrypted index construction, token-based search, deletion, password rotation and an explicit leakage model within one implementation.

3.1 Key Derivation and Vault Unlock

The security of every stored document depends on the key hierarchy, so the system never uses the password directly as a key. When the vault is first created, a random 16-byte salt is generated and stored on the cloud. On every unlock, the client combines the entered password with this salt using Argon2id, a memory-hard password-hashing function that makes large-scale guessing on GPUs or dedicated hardware expensive. The parameters follow RFC 9106 [14]: three iterations, 64 MiB of memory and four parallel lanes. The resulting 256-bit master key is never used directly; HKDF-SHA256 expands it into two independent sub-keys using distinct, versioned labels, so that compromise of one sub-key reveals nothing about the other. The derivation can be represented as

$$MK = Argon2id(pw, salt;\ t = 3,\ m = 64\ MiB,\ p = 4) \quad (1)$$

$$K_{file} = HKDF(MK, \text{“vault:k_file:v1”}),\quad K_{index} = HKDF(MK, \text{“vault:k_index:v1”}) \quad (2)$$

where pw is the user password, salt is the stored 16-byte random salt, MK is the 256-bit master key, K_{file} is the key used to encrypt document contents and filenames, and K_{index} is the root key from which all keyword keys are derived. Because SSE decryption of a wrong-key index would otherwise fail silently, a password verifier V is stored alongside the salt. It is the AES-GCM encryption of a fixed constant under K_{index}; on unlock the client must decrypt V successfully before the derived keys are accepted, so a wrong password is rejected before any key reaches the upload or index logic.

$$V = AES\text{-}GCM(K_{index}, \text{“vault-verifier:v1”}) \quad (3)$$

Step 1 – Master Key Derivation: The client fetches the non-secret salt from the cloud and derives the master key with Argon2id at the shipped parameters. The measured unlock time on the evaluation machine is approximately 57 ms, a delay that is negligible for a user but multiplies the cost of every password guess for an attacker.

Step 2 – Sub-Key Separation and Password Verification: The master key is expanded into K_{file} and K_{index} with HKDF-SHA256, and the stored verifier is decrypted with K_{index}. If authentication fails, the unlock request is rejected with an error and no key is retained; if it succeeds, both sub-keys are held only in the memory of the trusted client process for the session.

3.2 Document Encryption Module

Documents are protected with AES-256-GCM, an authenticated encryption mode that provides confidentiality and integrity in a single primitive. For every encryption a fresh 96-bit nonce is generated from the operating system’s secure random source, and no code path accepts a caller-supplied nonce, which structurally prevents the nonce reuse that would break GCM. The 128-bit authentication tag lets the client detect any modification of a stored ciphertext, and all decryption failures — wrong key or tampered bytes — are collapsed into one generic error so that no oracle-style side channel is created. The filename is encrypted separately from the contents, so the cloud sees neither. The encryption can be represented as

$$C = AES\text{-}GCM(K_{file}, N, M) = (N, CT, T) \quad (4)$$

where M is the plaintext document (or filename), N is the 96-bit random nonce, CT is the ciphertext and T is the 128-bit authentication tag. The stored ciphertext is therefore exactly 28 bytes longer than the plaintext (12-byte nonce and 16-byte tag), a value confirmed by measurement in Section IV.

Step 1 – Text Extraction and Identifier Assignment: The uploaded file is accepted as UTF-8 text, a PDF with a readable text layer, or a zip archive of either, and its readable text is extracted for indexing. A random 128-bit document identifier is generated, so identifiers reveal nothing about the document.

Step 2 – Content and Filename Encryption: The raw file bytes and the filename are each encrypted under K_{file} with independent nonces. Only the identifier, the encrypted filename, the nonce, the ciphertext and the tag are transmitted to the cloud, which stores them in its documents table.

3.3 Keyword Extraction and Encrypted Index Construction

The searchable index is the core of the scheme. Keywords are extracted with a single deterministic tokenizer that lower-cases the text, splits it into alphanumeric tokens, discards tokens shorter than two characters and common stop words, and removes duplicates within each document. Because one function is used both when building the index and when processing a query, index-time and search-time normalisation cannot diverge, so a search for "Encryption." matches a document containing "encryption". For each keyword w two keyword keys are derived from K_{index} with the pseudorandom function F = HMAC-SHA256:

$$K_1 = F(K_{index}, \text{“1”} \Vert w), \quad K_2 = F(K_{index}, \text{“2”} \Vert w) \tag{5}$$

where K_1 generates the lookup labels and K_2 encrypts the document identifiers. If DB(w) is the list of identifiers of documents containing w, the index stores one entry for every (keyword, document) pair, or posting:

$$\ell_c = F(K_1, c), \quad v_c = AES\text{-}GCM(K_2, id_c), \quad c = 0, 1, \ldots, |DB(w)| - 1 \tag{6}$$

where c is a per-keyword counter, ℓ_c is a 256-bit label stored as 64 hexadecimal characters, and v_c is the encrypted identifier of the c-th matching document. The index is a flat table of (label, value) pairs: it contains no keyword, no identifier in plaintext and no indication of which rows belong to the same keyword. Documents are processed in sorted order so that labels are reproducible, while values use fresh nonces so that two rebuilds of an unchanged posting do not produce identical ciphertext.

Step 1 – Corpus Tokenisation: All documents in the vault are decrypted on the client and tokenised, producing a mapping from each identifier to its keyword set. Postings are grouped by keyword to form DB(w) for every keyword.

Step 2 – Index Generation and Atomic Replacement: For every keyword the client derives K_1 and K_2, emits one label–value pair per posting, and sends the complete index to the cloud, which replaces its stored index in a single transaction. Because the construction is static, the whole index is rebuilt after every upload, deletion or password change; the cost of this design choice is measured in Section IV.

3.4 Search Token Generation and Encrypted Search

To search for a keyword w, the client normalises the query with the same tokenizer and derives K_1, which is the only key material that ever crosses the trust boundary. K_2 is derived locally at the moment it is needed and is never transmitted, because the cloud never decrypts anything. On receiving the token, the cloud regenerates candidate labels $\ell_0, \ell_1, \ldots$ and looks each up in its index, stopping at the first counter that is absent. It therefore recognises which opaque rows exist but never learns the keyword, the identifiers or the document contents. The cloud implements its own copy of the HMAC function with the standard library, so the cloud package imports nothing from the client package. For a conjunctive query of q keywords, the client issues q independent single-keyword searches and intersects the decrypted results:

$$R(w_1 \wedge w_2 \wedge \ldots \wedge w_q) = R(w_1) \cap R(w_2) \cap \ldots \cap R(w_q) \tag{7}$$

where R(w) is the set of document identifiers matching keyword w. The intersection is performed on the trusted client and stops early as soon as it becomes empty. This design required no change to the cloud or the index, but it reveals to the cloud each keyword's individual result set and the fact that the keywords were queried together; this co-occurrence leakage is recorded explicitly in the leakage model of Section 3.6.

Step 1 – Token Generation: The query is tokenised and, for each keyword, K_1 is derived and sent to the cloud in a search request containing nothing else.

Step 2 – Cloud Search and Client-Side Decryption: The cloud runs the label regeneration loop and returns the encrypted values it finds. The client derives K_2, decrypts the identifiers, intersects the sets for multi-keyword queries, and fetches and decrypts the filenames of the matching documents for display. The selected document can then be downloaded and decrypted with K_{file}.

3.5 Document Deletion and Password Rotation

Deletion is performed in a deliberately ordered way. The client first rebuilds and replaces the encrypted index without the target document, and only then asks the cloud to delete the ciphertext. If the order were reversed and the rebuild failed, the index would still contain labels pointing to a document that no longer exists. Password rotation is implemented as a full re-encryption, because every key in the hierarchy depends on the password. The client independently re-derives the keys from the current password and verifies them against the stored verifier, then performs two phases. In the validation phase the entire corpus is fetched, decrypted and tokenised under the old keys while nothing is written; if any single document fails, the operation is aborted with the vault unchanged. In the write phase a new salt and new keys are derived, every document is re-encrypted in place, the index is rebuilt under the new index key, and the new salt and verifier are written last.

Step 1 – Validate Before Writing: Every operation that could leave the vault partially updated (upload rebuild, deletion and rotation) first validates the whole affected corpus and writes nothing if any document cannot be processed.

Step 2 – Documented Limitation: The write phase of rotation is not atomic across documents. If the connection to the cloud is lost part-way, the vault can be left in a mixed state; this limitation is reported to the user and documented rather than hidden, and atomic rotation through envelope encryption is listed as future work.

3.6 Trust Boundary and Leakage Model

Every efficient SSE scheme reveals some information to the server, so the system defines precisely what the untrusted cloud learns and demonstrates each item with automated tests that capture the real HTTP traffic between the two processes. The cloud never receives the password, either sub-key, any keyword key K_2, any plaintext content, any plaintext filename or any plaintext keyword. It does learn the number of stored documents and their ciphertext sizes; the total number of postings, which is the size of the index; the search pattern, because the same keyword always produces the same token K_1; the access pattern, because it observes which index rows match a token and which documents are subsequently fetched by identifier; and, for conjunctive queries, the individual result set of each keyword and their co-occurrence. Because the index is static, every upload or deletion also causes the client to fetch every stored document during the rebuild, which the cloud can observe. These leakages are accepted as the price of efficient search and are stated as limitations rather than claimed to be absent. The complete architecture is shown in Fig. 1.

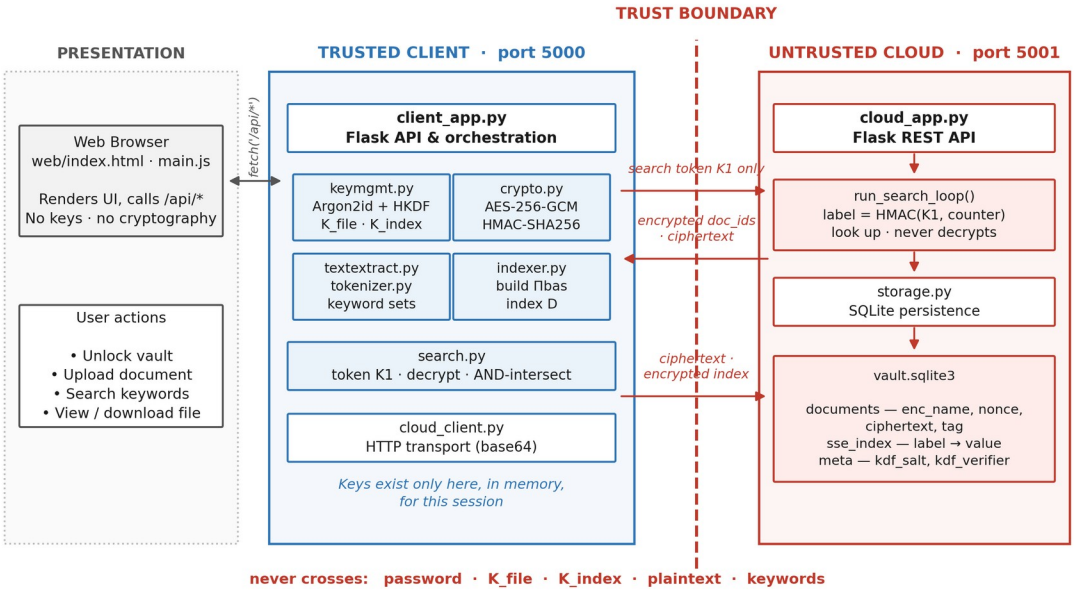


Fig. 1 Proposed SSE-based encrypted cloud storage system with a process-level trust boundary

3.7 SSE-Based Encrypted Storage and Search Algorithm

The proposed algorithm is designed to store documents on an untrusted server and retrieve them by keyword while keeping the documents, filenames and keywords confidential. It combines Argon2id and HKDF for key management, AES-256-GCM for document protection, and an HMAC-based encrypted inverted index for search. Instead of downloading and decrypting the whole vault for every query, the client sends a short search token and decrypts only the matching identifiers. The complete workflow is shown in Fig. 2.

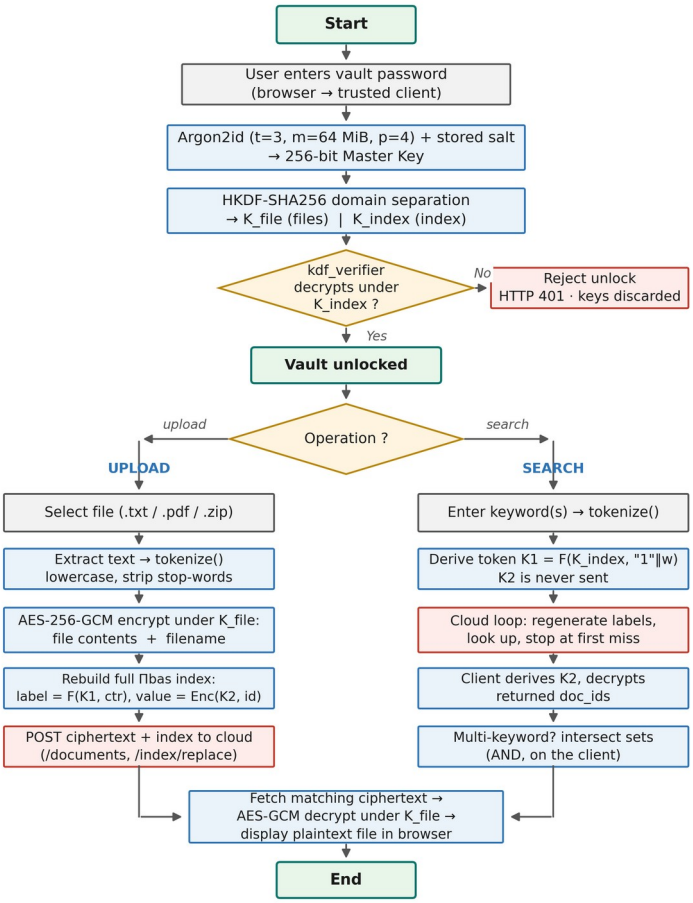


Fig. 2 Complete workflow of the implemented system

Input: Password pw, document set D = {D_1, D_2, …, D_n} and query keywords w_1, …, w_q

Output: Encrypted documents and encrypted index stored on the cloud, followed by the decrypted documents matching the query

Step 1 - Derive the keys

The password and the stored salt are processed with Argon2id to obtain the master key, which HKDF-SHA256 expands into K_{file} and K_{index} as in (1) and (2). The verifier (3) is decrypted to confirm that the password is correct.

Step 2 - Extract and tokenise each document

Readable text is extracted from each uploaded document and passed through the deterministic tokenizer to obtain its keyword set $W(D_i)$.

Step 3 - Encrypt the document and its filename

A random identifier id_i is assigned and the contents and filename are encrypted under K_{file} with fresh nonces, as in (4).

Step 4 - Upload the encrypted document

The identifier, encrypted filename, nonce, ciphertext and tag are sent to the cloud. The plaintext and the key never leave the client.

Step 5 - Derive the keyword keys

For every keyword w in the corpus, K_1 and K_2 are derived from K_{index} as in (5).

Step 6 - Build the encrypted index

For every posting of w, the label and encrypted value are computed as in (6). The resulting table of (label, value) pairs reveals neither keywords nor identifiers.

Step 7 - Replace the index on the cloud

The complete index is transmitted and replaced in a single transaction, so the cloud never holds a partially built index.

Step 8 - Generate the search token

For each query keyword the client normalises the keyword and sends only $T_w = K_1$. K_2 remains on the client.

Step 9 - Run the cloud search loop

The cloud regenerates labels from the token and collects the stored values until the first label that is absent:

$$S(w) = \{ v_c : \ell_c = F(T_w, c) \in I,\ c = 0, 1, 2, \dots \} \quad (8)$$

Step 10 - Decrypt the matching identifiers

The client derives K_2 and decrypts every returned value to recover the set of matching identifiers:

$$R(w) = \{ AES\text{-}GCM^{-1}(K_2, v) : v \in S(w) \} \quad (9)$$

Step 11 - Combine multi-keyword results

For a conjunctive query, the identifier sets of all query keywords are intersected on the client as in (7).

Step 12 - Retrieve and decrypt the documents

The matching encrypted documents are fetched by identifier and decrypted with K_{file}; the authentication tag is verified so that any modified ciphertext is rejected rather than returned.

$$M = AES\text{-}GCM^{-1}(K_{file}, N, CT, T) \quad (10)$$

IV. RESULT AND DISCUSSION

4.1: Simulation Tools

The proposed system was implemented in Python 3.13 using Flask 3.1.3 for the two web services, the *cryptography* library 50.0.1 for AES-256-GCM, HMAC-SHA256 and HKDF-SHA256, *argon2-cffi* 25.1.0 for Argon2id, *pypdf* for PDF text extraction, SQLite 3.50.4 for cloud storage, and *pytest* for automated testing. Functional results were obtained with an eight-document hand-written sample corpus. Performance was measured on a Windows 11 laptop with a 64-bit Intel processor (10 physical and 12 logical cores) and 16 GB of RAM, using a deterministic synthetic corpus whose word frequencies follow a Zipf-like distribution with roughly 40% stop words, so that the 10-document corpus is an exact prefix of the 200-document corpus and

every run indexes identical text. Each timing point was repeated (n = 10 to 30 after warm-up) and is reported as the median. The system was evaluated on the following parameters:

Correctness: Exact match between search results and the corpus ground truth, and successful decryption after retrieval

Search Latency: Cloud-side search time and client end-to-end search time

Processing Time: Key derivation, encryption, decryption, index construction and index rebuild time

Storage Overhead: Size of the encrypted database relative to the plaintext

Key Size: AES-256-GCM with 256-bit keys, HMAC-SHA256 with 256-bit labels, Argon2id with 64 MiB memory cost

4.2: Functional Results

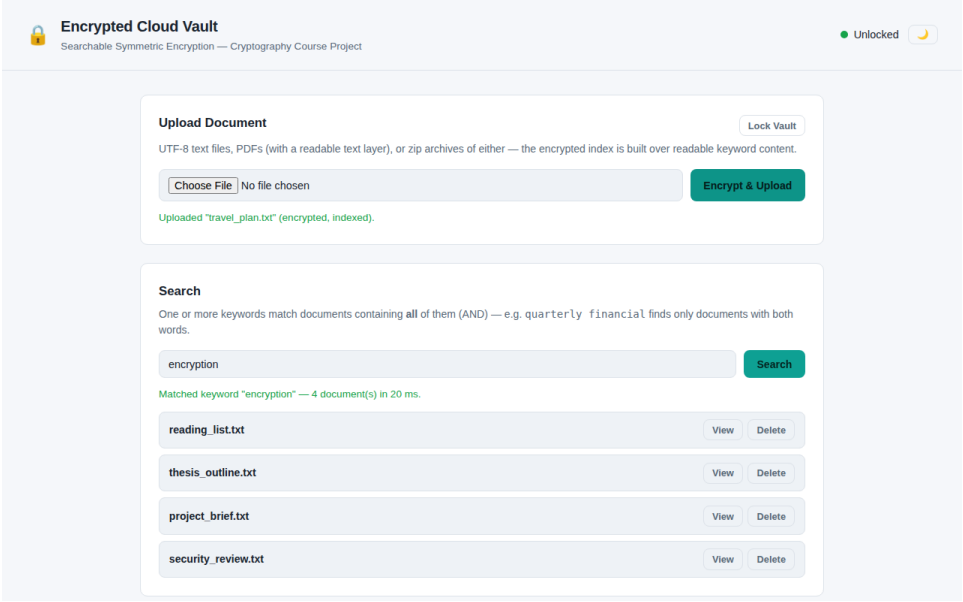


Fig. 3 Single-keyword encrypted search returning four matching documents

Fig. 3 shows a single-keyword search for “encryption” over the eight encrypted sample documents, whose index contains 400 postings. The query returned exactly the four documents that contain the keyword in 20 ms, matching the ground truth obtained by running the tokenizer over the plaintext files.

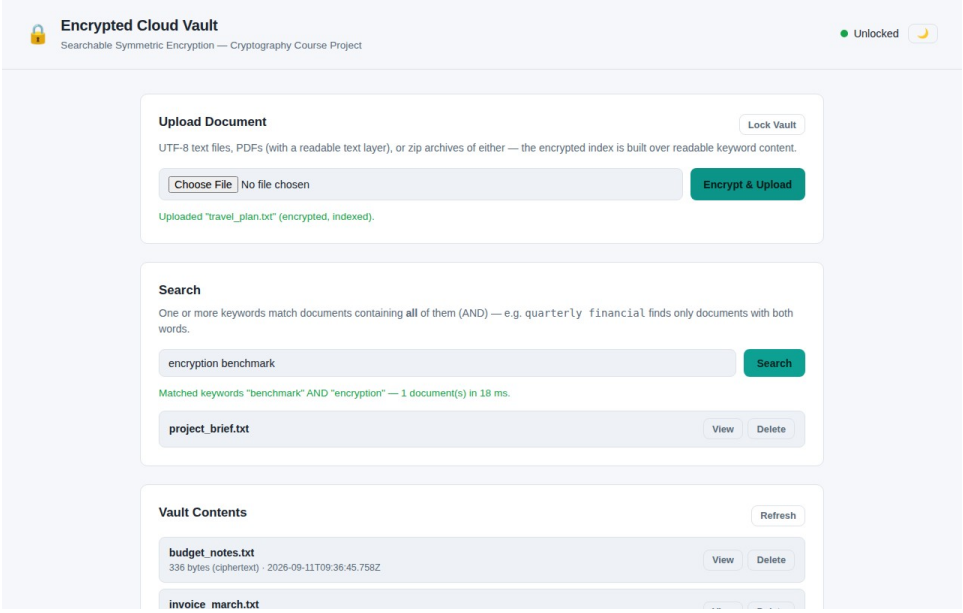


Fig. 4 Conjunctive (AND) search intersecting two keyword result sets on the client

Fig. 4 shows the conjunctive query “encryption benchmark”. Each keyword matches four documents, and the client-side intersection correctly returned the single document containing both terms in 18 ms. The captured traffic for this query contains two independent search requests, each carrying only one token.

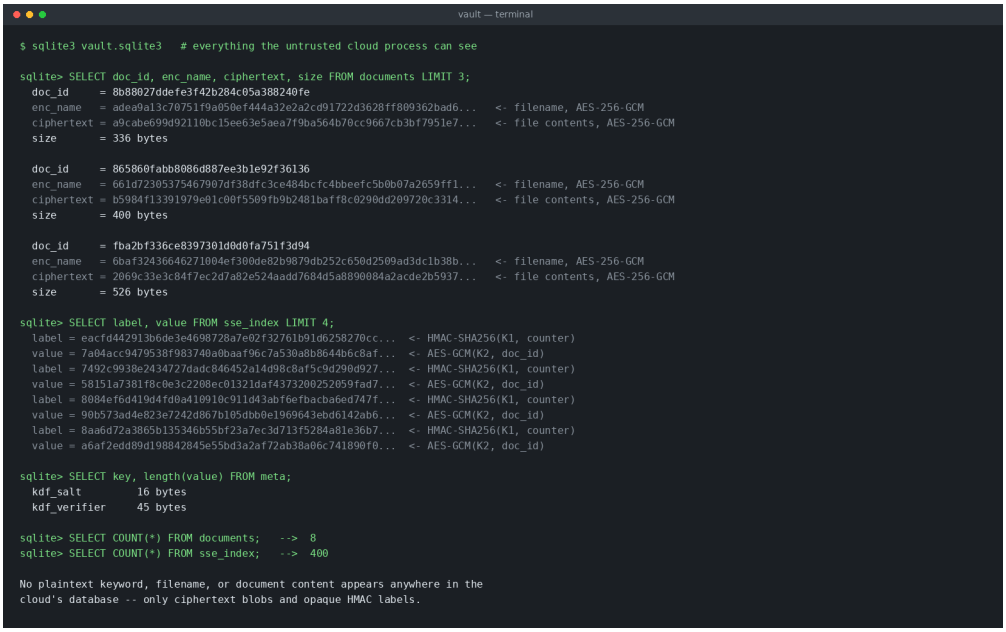


Fig. 5 Contents of the cloud database — only ciphertext and opaque HMAC labels are stored

Fig. 5 shows the contents of the cloud database. No plaintext filename, keyword or document content is present; the cloud holds only AES-GCM ciphertext, 64-character HMAC labels, encrypted identifiers, the non-secret salt and the password verifier, confirming the trust boundary of Section 3.6.

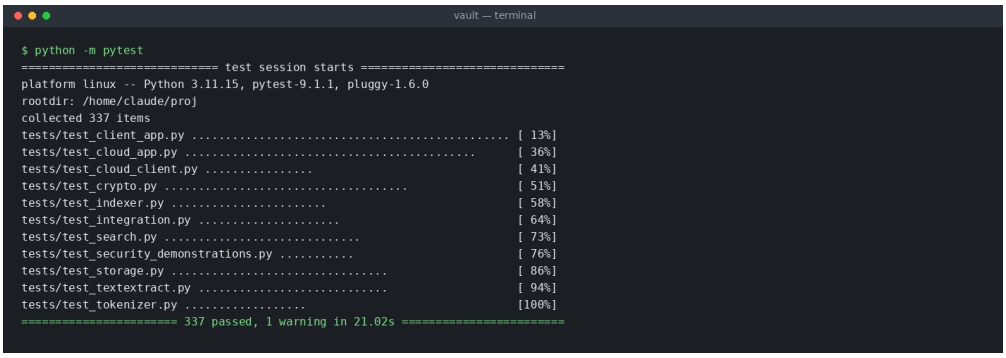


Fig. 6 Complete automated test suite passing (337 tests)

Fig. 6 shows the complete automated test suite passing with 337 tests covering the cryptographic primitives, the tokenizer and index, the search protocol, both web services, deletion, password rotation and the security demonstrations that capture real client–cloud traffic to verify the leakage model.

4.3: Performance Evaluation

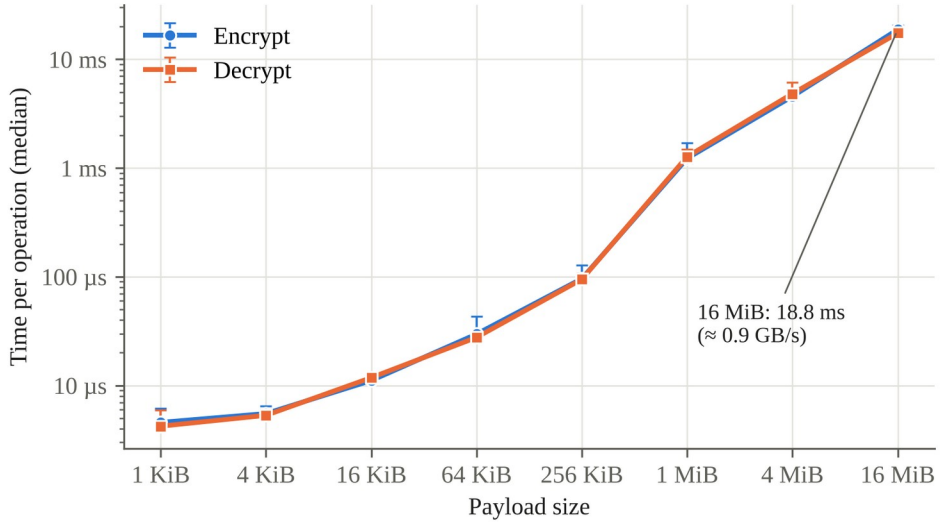


Fig. 7 AES-256-GCM encryption and decryption time versus payload size (median, min–max bars, n = 15)

Fig. 7 illustrates the cost of document encryption. Encryption and decryption track each other closely and scale linearly with payload size: a 1 MiB document is encrypted in a median of 1.23 ms and a 16 MiB payload in 18.8 ms, corresponding to roughly 0.9 GB/s. Every ciphertext is exactly 28 bytes longer than its plaintext. Symmetric encryption is therefore not a bottleneck of the system; the deliberately slow operation is key derivation, where a complete Argon2id unlock takes a median of 57 ms, while each HMAC evaluation takes about 3.4 µs and generating a complete search token about 5 µs.

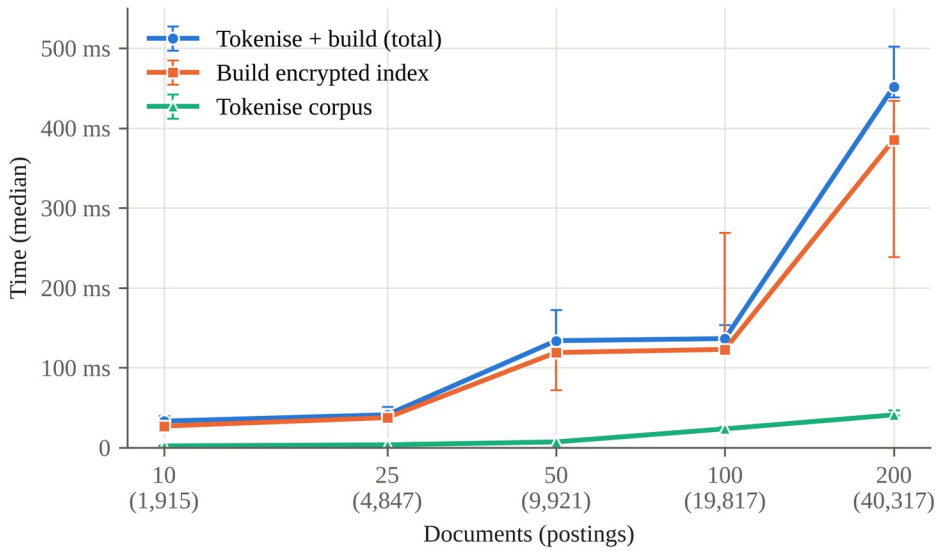


Fig. 8 Encrypted index construction time versus number of documents (median, min–max bars, n = 10)

Fig. 8 shows the cost of building the encrypted index on the client. For 10 documents (1,915 postings) tokenisation and index construction together take 33 ms, and for 200 documents (40,317 postings over 6,074 distinct keywords) they take 452 ms, of which 386 ms is index construction and only 41 ms tokenisation. The cost follows the number of postings rather than the number of documents, at roughly 11 µs per posting for the 200-document corpus, because each posting requires one HMAC label and one AES-GCM encryption. The curve is not perfectly smooth: the 50- and 100-document medians are close and their ranges are wide, so intermediate points should be read as approximate.

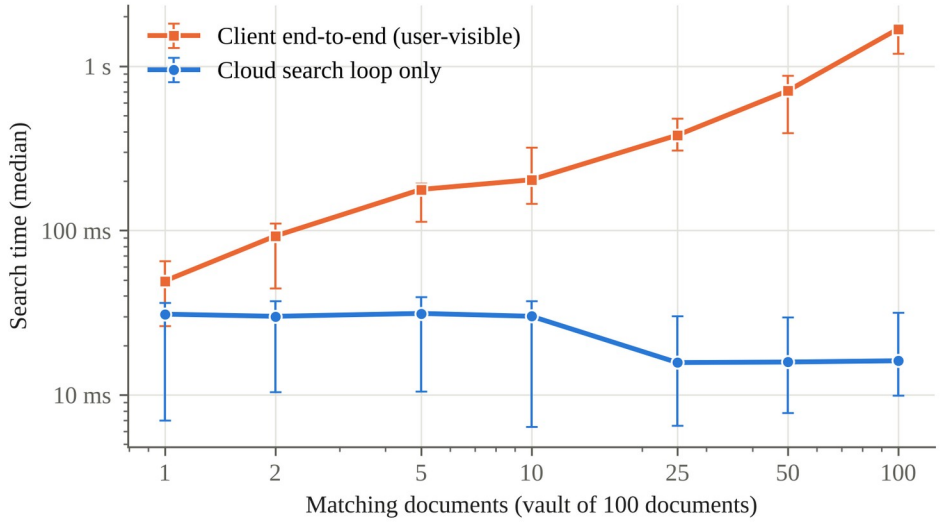


Fig. 9 Cloud-side search time and client end-to-end search latency versus number of matching documents

Fig. 9 compares the time spent by the cloud in the search loop with the latency experienced by the user, for a vault of 100 documents. The cloud-side time stays within an approximately flat band of 16–31 ms as the number of matching documents grows from 1 to 100, whereas end-to-end latency grows almost linearly, from 49 ms for one match to 1.68 s for 100 matches. The captured traffic shows exactly one document request per result, issued to retrieve each encrypted filename for display, so the bottleneck is this per-result fetch pattern rather than the cryptographic search, and batching these requests is the most effective optimisation available.

In a separate experiment with five matches, increasing the vault from 25 to 200 documents left the cloud search time at 15–17 ms and the end-to-end time at 96–98 ms, showing that search cost does not depend on the size of the vault.

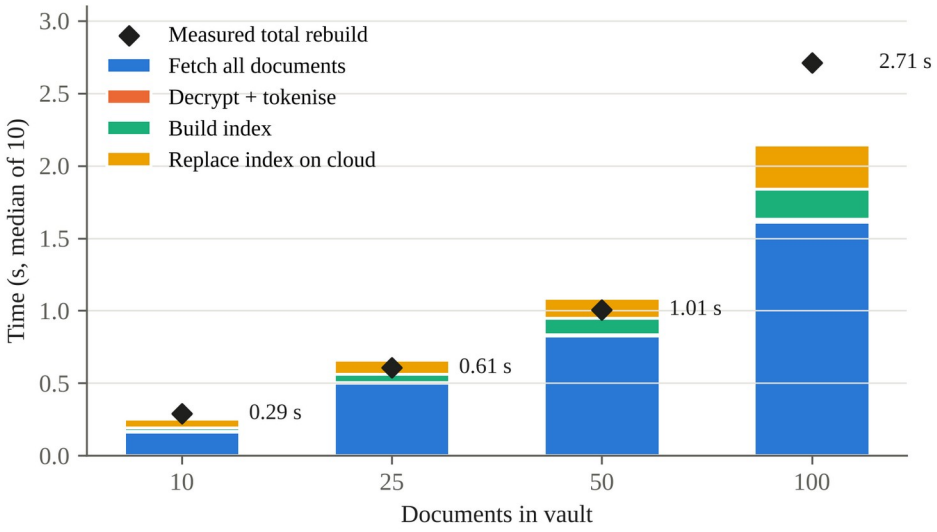


Fig. 10 Decomposition of the static index rebuild time performed after every upload

Fig. 10 decomposes the full index rebuild that follows every upload. The median rebuild time rises from 0.29 s at 10 documents to 2.71 s at 100 documents. At 100 documents, fetching every stored document from the cloud accounts for 1.61 s of the 2.15 s summed step time, whereas decryption and tokenisation take 22 ms, index construction 208 ms and index replacement 305 ms. The rebuild cost is therefore dominated by client–cloud transfer and grows with the vault size, which is the main practical limitation of the static design. Rebuild cost is reported only up to 100 documents, because none of the three benchmark runs reached the 200-document checkpoint.



Fig. 11 Storage footprint of the encrypted database at 200 documents (single snapshot)

Fig. 11 shows the storage footprint for a single 200-document snapshot. The plaintext content and filenames occupy 944,350 bytes, whereas the encrypted SQLite database occupies 10,039,296 bytes, an expansion of

about 10.6 times. The document ciphertext itself adds only 28 bytes per document; most of the growth comes from the encrypted index, whose 40,317 postings require about 5.0 MB (124 bytes per posting for a 64-character label and a 60-byte encrypted identifier), and from about 4.1 MB of database structural overhead derived as the difference between the file size and the measured column data. The storage cost of searchability is therefore significant and is driven by the number of postings rather than by the document data. Because storage was measured at one corpus size only, this figure is not a scaling result.

V. CONCLUSION

This work presented the design, implementation and evaluation of an encrypted cloud storage system based on searchable symmetric encryption, in which a trusted client and an untrusted cloud run as separate processes and only ciphertext, opaque labels and per-keyword search tokens cross the trust boundary. The system supports password-based key derivation with Argon2id and HKDF, AES-256-GCM document and filename encryption, an HMAC-based static encrypted index, single- and multi-keyword search, deletion and password rotation, and is verified by 337 automated tests. The evaluation showed that the cryptographic operations are inexpensive and that cloud-side search time is approximately constant with respect to both the number of results and the size of the vault, whereas user-visible latency is dominated by per-result document requests and update cost by the full static index rebuild. The encrypted index increases storage by roughly an order of magnitude. Future work includes batching result retrieval, replacing the static index with a dynamic scheme offering forward and backward privacy, reducing search- and access-pattern leakage, making password rotation atomic through envelope encryption, and extending the system to multiple authorised users.

References

[1] Z. Hu, J. Wang, Z. Chen, Z. Ge, M. Bian, L. Chen, and Y. Zhou, “SEAC: Dynamic Searchable Symmetric Encryption with Lightweight Update-Search Permission Control,” Cybersecurity, vol. 8, article 75, 2025.

[2] S. Xu, X. Chen, Y. Guo, Y. Yang, S. Wang, S.-M. Yiu, and X. Cheng, “Forward-Secure Multi-User and Verifiable Dynamic Searchable Encryption Scheme within a Zero-Trust Environment,” Future Generation Computer Systems, vol. 166, p. 107701, 2025.

[3] L. Yang, Y. Yang, D. Niyato, Z. Li et al., “Dynamic Searchable Symmetric Encryption With Efficient and Complete Access Control for Multi-User Cloud Computing,” IEEE Transactions on Mobile Computing, 2025.

[4] “Forward and Backward Private Conjunctive Dynamic Searchable Symmetric Encryption With Refined Leakage Function and Low Communication,” IEEE Transactions on Information Forensics and Security, vol. 20, pp. 8224–8236, 2025.

[5] Z. Ren, P. He, R. Yu, L. Deng, Y. Tong, and S. Xu, “Dynamic Searchable Symmetric Encryption with Efficient Conjunctive Query and Non-Interactive Real Deletion,” Journal of Network and Computer Applications, vol. 245, article 104387, 2026.

[6] X. Zhang, H. Wang, J. Lu, T. Li et al., “A Bloom Filter-Based Dynamic Symmetric Searchable Encryption Scheme over Cloud Data,” Cybersecurity, vol. 9, article 47, 2026.

[7] X. Wang, J. Gong, K. Zhang, and H. Qian, “Decentralized Multi-Client Boolean Keyword Search for Encrypted Cloud Storage,” Computer Standards & Interfaces, vol. 97, article 104127, 2026.

[8] P. Wu, J. Shen, Z. Cao, and X. Dong, “MMKFB: Multi-Client and Multi-Keyword Searchable Symmetric Encryption with Forward and Backward Privacy,” Frontiers of Computer Science, vol. 19, no. 3, article 193804, 2025.

[9] A. Wu, D. Feng, M. Zhang, J. Chi, and Y. Zhang, “Efficient Verifiable Searchable Encryption with Search and Access Pattern Privacy,” Security and Safety, vol. 4, article 2024022, 2025.

[10] A. N. Khan, A. Naveed, A. Mehmood, D. Arora, A. ur R. Khan, and J. Ali, “BloomSec: Scalable and Privacy-Preserving Searchable Encryption for Cloud Environments,” PLOS ONE, vol. 20, no. 12, article e0336944, 2025.

[11] J. Xue, L. Zeng, W. Zhang, F. Li, X. Zhang, Y. Zhou, and L. Liu, “Incentive-Assisted Multi-Keyword Ranked Searchable Encryption with Hidden Attributes for Community Data Sharing,” Peer-to-Peer Networking and Applications, vol. 18, no. 3, article 153, 2025.

[12] H. M. Mohammed, M. H. Hashim, D. Kh. Altmemi, Z. A. Abduljabbar, Z. A. Hussien, V. O. Nyangaresi, A. A. Alyousif, and H. A. Neamah, “A Review: Searchable Symmetric Encryption Functionalities, Challenges and Solutions,” in Software Engineering: Emerging Trends and Practices in System Development, Lecture Notes in Networks and Systems, vol. 1562, Springer, Cham, pp. 47–65, 2026.

[13] D. Cash, J. Jaeger, S. Jarecki, C. Jutla, H. Krawczyk, M.-C. Roşu, and M. Steiner, “Dynamic Searchable Encryption in Very-Large Databases: Data Structures and Implementation,” in Proc. Network and Distributed System Security Symposium (NDSS), 2014.

[14] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, “Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications,” RFC 9106, Internet Engineering Task Force, Sept. 2021.